

# Lab Manual — Week 7

*Concurrency & Distributed Systems* — Lebanese University, Faculty of Engineering

**Topic:** Futures, Async Design, Testing Traps, and Anti-Patterns

## Learning Objectives

By the end of Week 7, you will understand:

1. **CompletableFuture pipelines:** How to compose asynchronous service calls.
2. **Timeouts:** How to avoid waiting forever for a slow dependency.
3. **Fallbacks:** How to return a controlled value when a service is slow or fails.
4. **Exception handling:** How to recover from asynchronous failures without hiding them.
5. **Fan-out/fan-in:** How to start independent calls and aggregate their results.
6. **Testing traps:** Why races may disappear in ordinary tests.
7. **NIO counterpoint:** Why async design is not the same as using more threads.

## Required Course Artifacts

You must submit:

- Completed exercise source files under `src/main/java/.../exercises`.
- Passing output from `gradlew.bat studentCheck`.
- Completed `CONCURRENCY_SCORECARD.md`.
- Completed fan-out/fan-in sequence diagram notes in `ASYNC_PIPELINE_SEQUENCE.md`.

## Lab Folder

```
week7-lab/week7-day1-async-testing-antipatterns
```

Use:

```
.\gradlew.bat test
.\gradlew.bat studentCheck
.\run-all-demos.ps1
```

## Part 1 — Async Service Simulation

Open:

```
part1_completable_future/Demo01_CompletableFuturePipeline.java
part1_completable_future/Demo02_TimeoutFallback.java
part1_completable_future/exercises/Ex1_AsyncTimeoutFallback.java
```

**Task:** Implement an async service call that returns:

- "real-value" when the service completes before the timeout,
- "fallback-value" when the service exceeds the timeout.

**Design rule:** The fallback must be explicit. Do not let the caller wait forever.

## Part 2 — Testing Trap

Open:

```
part2_testing_traps/Demo03_PassesUntilAmplified.java
part2_testing_traps/Demo04_ReproductionStrategy.java
part2_testing_traps/exercises/Ex2_ReproductionStrategy.java
```

**Task:** Return a reproduction strategy that uses:

- at least 10,000 iterations,
- yielding inside the critical window,
- randomized scheduling.

**Design rule:** A single passing test run does not prove thread safety.

## Part 3 — Stress-Style Classification

Open:

```
part3_stress_testing/Demo05_StressOutcomeClassification.java
part3_stress_testing/exercises/Ex3_StressCounterCheck.java
```

**Task:** Classify outcomes:

| Observed result                     | Classification |
|-------------------------------------|----------------|
| Equal to expected                   | Acceptable     |
| Less than expected but non-negative | Interesting    |
| Negative or greater than expected   | Forbidden      |

## Part 4 — Fan-Out / Fan-In Pipeline

Open:

```
part4_fan_out_fan_in/Demo06_FanOutFanInPipeline.java
part4_fan_out_fan_in/exercises/Ex4_FanOutFanInPipeline.java
```

**Task:** Build an async pipeline with these branches:

- **profile:** fast success,
- **pricing:** fast success,
- **inventory:** slow service that should become timeout fallback,
- **recommendations:** failing service that should become exception fallback.

Use `CompletableFuture.supplyAsync`, `CompletableFuture.allOf`, and explicit aggregation.

**Required result:** The combined response must contain four service results and must expose that fallback happened.

## Part 5 — NIO Counterpoint

Open:

```
part5_nio_counterpoint/Demo07_NioVsBlockingCounterpoint.java
part5_nio_counterpoint/exercises/Ex5_NioCounterpoint.java
```

**Task:** Explain why NIO can serve many waiting connections with fewer blocked threads, while async does not automatically make CPU work faster.

## Anti-Pattern Checklist

Before submitting, check that your answer avoids:

- thread-per-request as the only scalability story,
- unbounded queues,
- missing timeouts,
- silent failure swallowing,
- vague fallback values that hide degraded service behavior.

## Submission Checklist

- ☐ `gradlew.bat studentCheck` passes.
- ☐ Every async branch has a completion path.
- ☐ Timeout and fallback behavior is visible in the code.
- ☐ Fan-out/fan-in sequence diagram is completed.
- ☐ Scorecard is completed honestly.

## Bridge to Week 8

Week 7 teaches composition of asynchronous work. Week 8 will move toward server design and non-blocking I/O. Keep the distinction clear: fewer blocked threads can improve scalability, but correctness still depends on explicit failure, timeout, and completion rules.